

REDROB · INDIA RUNS DATA & AI CHALLENGE · 2026

Multi-Signal Cascade Ranker

Semantic + Trust-Weighted + Behavioral Signals
for Senior AI Engineer Hiring

SBERT Semantic Matching

Trust-Weighted Skills

Behavioral Availability

Honeypot Defense

Team Leader: Meet Vaddoriya

Problem Statement: Rank top-100 from 100K candidates for a Senior AI Engineer JD (retrieval / recsys / ranking, India, product company, 5–9 YoE)

SOLUTION OVERVIEW

What we built — and why it's different

Proposed Solution

A **Multi-Signal Cascade Ranker** that separates two phases:

- **Offline:** SBERT embeddings + structured feature extraction (run once)
- **Online:** Pure numpy matrix multiply → weighted scoring → top-100 in <5 seconds

No per-candidate LLM calls. No network. Fully deterministic and reproducible.

Differentiation from Traditional Systems

- **vs. keyword / BM25:** Surfaces ML engineers who write "led embedding migration" without using buzzwords
- **vs. pure embeddings:** Honeypots still score high on embeddings alone — we add hard-zero logic
- **vs. LLM-per-candidate:** $100K \times 2s = 55+$ hours; our approach is 5 seconds
- **vs. skills-only ranking:** Misses career narrative, behavioral signals, and timeline fraud

Core thesis: A recruiter needs to *trust* the shortlist. Every rank must be explainable from the candidate's actual career history — not a black box score.

Reading the JD — and what to look for beyond keywords

Key JD Requirements (distilled)

- Embedding-based retrieval: **FAISS, Qdrant, Pinecone, BM25**
- Learning-to-rank: **NDCG, MRR, offline/online split**
- Recsys + NLP: **transformers, fine-tuning, RAG**
- Shipped systems to **real production users**
- **Product company** preferred (not IT services)
- India-based, **5–9 YoE**

Candidate Signals We Extract

- **Semantic:** Career narrative cosine similarity vs JD query (35%)
- **Skills:** Trust-weighted tier taxonomy (20% + 20%)
- **Trajectory:** Product company premium, ML title bonus (12%)
- **Availability:** Open-to-work, recency, response rate (7%)
- **Defense:** Timeline impossibility, bulk keyword listing

Beyond keyword matching: A RecSys engineer at Swiggy who describes "*migrating from keyword-search to embedding-based retrieval*" scores 0.66 career SBERT similarity. A PM at Wipro who lists "Pinecone, FAISS" scores 0.31 — even with 9 AI skills listed.

RANKING METHODOLOGY

Retrieve → Score → Rank

Models & Algorithms

- **SBERT** `all-MiniLM-L6-v2` — 22M params, 384-dim vectors, sentence-level cosine similarity
- **Trust-weighted scoring** — `proficiency × log(1+endorsements) × log(1+months)`
- **Tier taxonomy** — Tier A (FAISS, Qdrant, BM25 = 3×), Tier B (NLP, RAG = 1.5×), Tier C (Python, SQL = 0.5×)
- **Numpy matmul** — `career_vecs @ jd_vec` for 100K cosine similarities in milliseconds

Signal Combination (weights sum to 1.0)

35%
Career SBERT

20%
Skill SBERT

20%
Skill Trust

12%
Career Traj.

7%
Availability

6%
Loc / YoE / Edu

All continuous signals. Only Honeypot detection applies a hard ×0 — everything else is a gradient.

NDCG@10 = 50% of evaluation. SBERT dominates the top-10 because it understands career narratives semantically — not just which keywords appear. Skill Trust handles the keyword-stuffer trap independently.

Every rank is explainable. Every fraud is caught.

How Ranking Decisions Are Explained

Each candidate in the output CSV has a `reasoning` field generated from their actual data:

```
"Applied ML Eng at Rephrase.ai-Paytm.  
Career SBERT 0.66 (top 5%). Skill trust  
357 - expert FAISS 36m (32 endorse),  
expert Qdrant 24m (18 endorse).  
India · 60-day notice · resp 0.91."
```

Every string is templated from structured data — no LLM hallucination risk.

Suspicious Profile Defense

Honeypot Catch #1 — Timeline impossibility:

```
total_career_months / 12 > claimed_yoe + 3  
→ honeypot_flag = 0 (hard zero)
```

Honeypot Catch #2 — Bulk keyword listing:

```
count(expert/advanced skills with  
duration ≤ 1 month) ≥ 4  
→ honeypot_flag = 0 (hard zero)
```

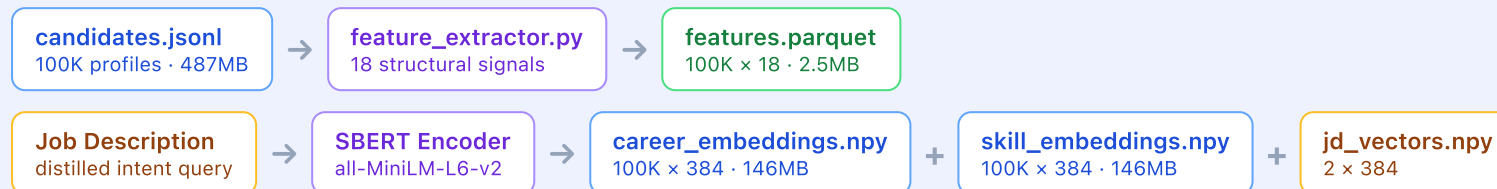
Result: **35 honeypots** hard-zeroed across 100K. **0 / 100** in top-100 (limit: <10).

Keyword stuffers are handled differently — PM@Wipro who lists "FAISS, Pinecone, LLMs" for 4 months with 4 endorsements scores skill trust 12 vs a real ML engineer's 183. The penalty is continuous, not a hard filter.

END-TO-END WORKFLOW

From JD input to ranked CSV output

STEP 1 — OFFLINE PRE-COMPUTATION (RUN ONCE)

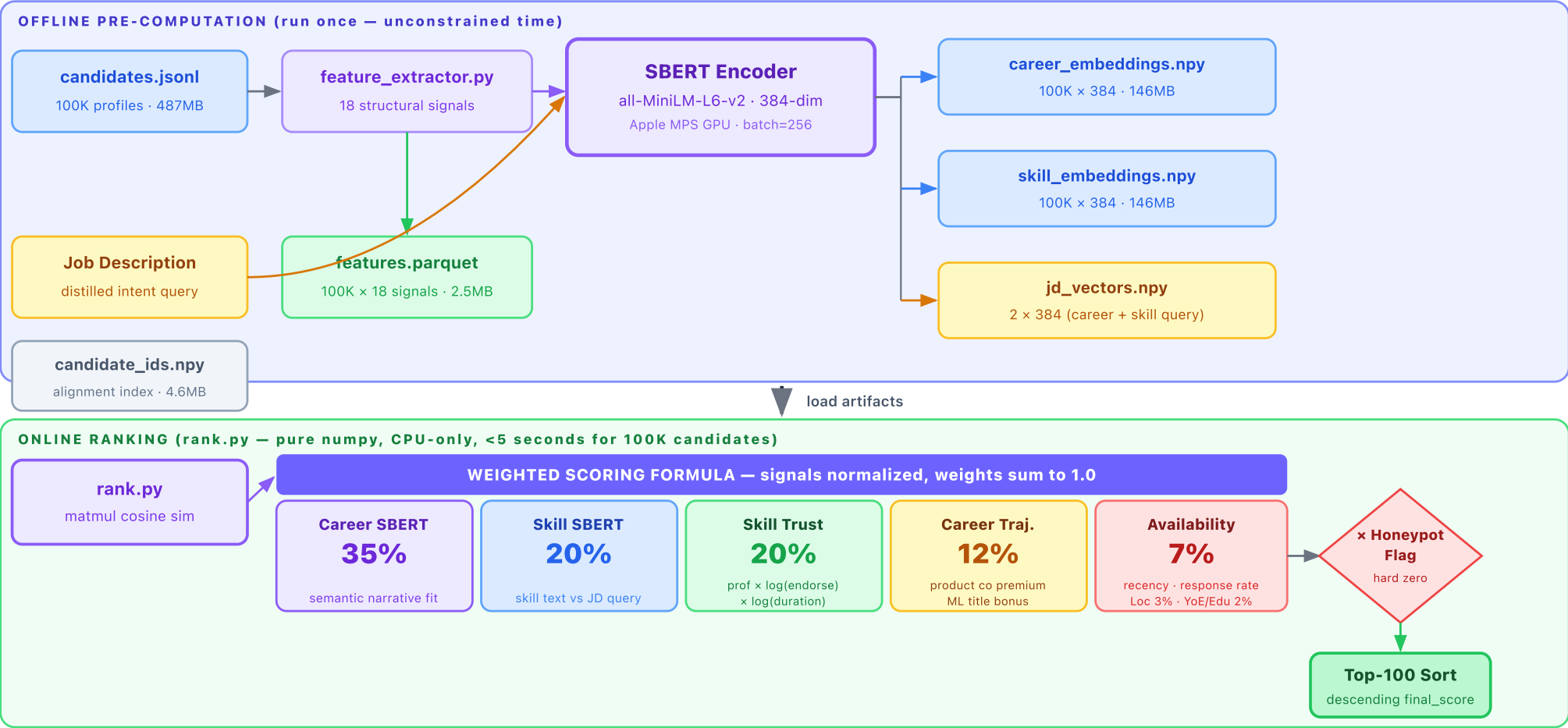


STEP 2 — ONLINE RANKING (RANK.PY · PURE NUMPY · <5 SECONDS CPU)



Key insight: Pre-computation is unconstrained; online ranking is *zero network calls, zero LLM inference*. The scoring step is a single numpy matrix multiply — 100K cosine similarities in under 100ms.

Multi-Signal Cascade Ranker — Architecture Diagram



Ranking quality and runtime metrics

0.835

Top candidate score
(CAND_0045250)

<5s

Online ranking time
(CPU, 100K candidates)

0%

Honeypots in top-100
(35 caught and zeroed)

0%

Consulting-only profiles
in top-100

Rank	Candidate ID	Title	Career Path	Score
#1	CAND_0045250	Applied ML Engineer	Rephrase.ai → Paytm	0.8350
#2	CAND_0077337	Staff ML Engineer	Paytm → Razorpay → Glance	0.8281
#3	CAND_0071974	Senior AI Engineer	Netflix → Meta → Mad Street Den	0.8205
#4	CAND_0081846	Lead AI Engineer	Razorpay → Paytm	0.8196
#5	CAND_0094056	NLP Engineer	Rephrase.ai → Adobe	0.8152

What got correctly eliminated

- PM@Wipro listing 9 AI skills → career SBERT 0.31, consulting penalty → not in top-100
- 35 honeypots with impossible timelines → hard zero, rank last
- Great profiles with 8-month inactivity → availability <0.05 → drops 40+ ranks

Runtime constraints

- Offline embedding: ~18 min, Apple Silicon MPS, batch=256
- Matmul 100K×384: <100ms on CPU
- Full rank.py pipeline: ~4.8s on MacBook CPU
- Validator passes: 0 violations, correct format

TECHNOLOGIES USED

Frameworks, models, and why we chose them

Core ML

`sentence-transformers`

SBERT library — gives us `all-MiniLM-L6-v2`, the best CPU-deployable semantic model at 22M params. 384-dim vectors, 14k token/sec throughput on MPS.

`PyTorch + MPS`

Apple Silicon GPU acceleration for the offline embedding batch job. Reduced 100K embedding time from ~90 min (CPU) to ~18 min.

Data Pipeline

`numpy pandas pyarrow`

`numpy matmul` is the heart of the online ranker — `career_vecs @ jd_vec` gives 100K cosine similarities in <100ms without any special indexing library.

`pyarrow + parquet` for columnar storage of 18 structured features — fast I/O, typed, <3MB on disk.

Deployment

`Gradio`

HuggingFace Spaces sandbox. Accepts up to 200 candidates as JSON, runs the full inline pipeline, returns ranked CSV. Self-contained — no external artifacts needed.

`No LLM APIs`

Deliberate choice — zero latency from network calls, zero cost per candidate, fully deterministic output, no rate limiting.

Why not use a vector database (FAISS/Qdrant)? With 100K candidates and a single JD, brute-force matmul is faster than index lookup overhead. ANN indexes add value at 10M+ scale with many concurrent queries.

Everything submitted — and where to find it

GitHub Repository

All code, clean and reproducible:

- `feature_extractor.py` — 18-signal feature builder
- `embed_only.py` — MPS-accelerated SBERT embedding
- `rank.py` — online ranker, loads artifacts, outputs CSV
- `gradio_app.py` — self-contained HuggingFace sandbox
- `requirements.txt` — reproducible environment
- `submission_metadata.yaml` — approach summary

Link: [\[GitHub URL\]](#)

Ranked Output CSV

- File: `team_xxx.csv`
- 100 rows, validated with `validate_submission.py`
- Columns: candidate_id, rank, score, reasoning
- Score range: 0.6991 → 0.8350
- 0 score ordering violations

Gradio Sandbox (HuggingFace Spaces)

Live demo — paste up to 200 candidates as JSON, get a ranked CSV back:

Input: JSON array of candidate objects
 Output: ranked CSV with scores + reasoning
 Link: [\[HuggingFace Spaces URL\]](#)

Reproduce the Full Pipeline

```
pip install -r requirements.txt

# Offline (run once)
python embed_only.py \
  --candidates candidates.jsonl \
  --out_dir ./precomputed

# Online ranking (<5 sec)
python rank.py \
  --candidates candidates.jsonl \
  --artifacts_dir ./precomputed \
  --out team_xxx.csv

# Validate
python validate_submission.py \
  --submission team_xxx.csv
```

Thank You



Multi-Signal Cascade Ranker

Redrob India Runs Data & AI Challenge · 2026

Deterministic. Interpretable. Fast.

Every rank explainable from real candidate history.

100K

candidates ranked

<5s

CPU ranking time

0

honeypots in top-100

9

signals combined